

A Beginner's Guide to Handling Missing Values in Pandas

Introduction

In real-world datasets, missing values are very common. Machine learning models cannot work properly with missing values, meaning we must handle them before training our models.

1 How to Identify Missing Values

In the Python library Pandas, missing values are usually represented as:

- **NaN** (Not a Number)
- **None**
- Empty cells

Example Dataset Output:

Name	Age	Department	Salary
Rahul	25	IT	50000
Priya	NaN	HR	45000
Arjun	28	NaN	60000
Sneha	35	IT	NaN

2 Detecting Missing Values

Before handling missing values, we must locate them. We can check for missing data using the following Pandas functions:

```
1 import pandas as pd
2
3 # Check for missing values in the entire dataframe
4 print(df.isnull())
5
6 # Count missing values per column (Highly Recommended)
7 print(df.isnull().sum())
```

3 Handling Missing Values: Deletion

If a row or column has too much missing data, the simplest approach is to remove it entirely.

Drop rows with any missing values:

```
1 df_cleaned = df.dropna()
```

Drop columns with missing values:

```
1 df_cleaned = df.dropna(axis=1)
```

4 Handling Missing Values: Imputation (Filling)

Instead of deleting data, we can estimate and fill in the missing values.

1. Fill with a constant value:

```
1 # Fill all missing ages with 0
2 df['Age'] = df['Age'].fillna(0)
```

2. Fill with Mean / Median (For Numerical Data):

```
1 # Fill missing salaries with the average salary
2 df['Salary'] = df['Salary'].fillna(df['Salary'].mean())
```

3. Fill with Mode (For Categorical/Text Data):

```
1 # Fill missing departments with the most frequent department
2 df['Department'] = df['Department'].fillna(df['Department'].mode()[0])
```